PROMPT LOG

Prompts Used to Build SQL Practice

Build an Interactive SQL Practice Website

1. Product Goal Build a polished, modern web application where anyone can learn and practice SQL from Beginner → Intermediate → Advanced. The website should be completely self-contained:

- Users should not need to provide their own database.
- All practice tables and sample data should already exist inside the application.
- Users should have an online SQL editor/compiler where they can write and execute queries.
- Every question should have an expected result/query validation system.
- Users should receive immediate feedback after running their query.
- The experience should feel like a dedicated SQL learning platform, not a generic coding playground.

The primary goal is to help users develop SQL problem-solving ability, not just memorize syntax. 2. Target Users The platform should work for:

- Complete SQL beginners
- People preparing for SQL interviews
- Software engineers
- QA engineers
- Data analysts
- Backend developers
- Students
- Experienced SQL users who want advanced practice

Users should be able to start from zero and gradually progress to advanced SQL. 3. Learning Levels Create three main levels: Beginner Topics should include:

1. SELECT
2. DISTINCT
3. WHERE
4. AND / OR
5. IN
6. NOT IN
7. BETWEEN
8. LIKE
9. IS NULL / IS NOT NULL
10. ORDER BY
11. LIMIT
12. Aliases
13. Basic string functions
14. Basic date functions
15. COUNT
16. SUM
17. AVG
18. MIN
19. MAX
20. GROUP BY
21. HAVING

Include approximately 30-40 beginner questions. Intermediate Topics should include:

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. Multiple table joins
5. UNION
6. UNION ALL
7. INTERSECT
8. MINUS / EXCEPT depending on SQL engine
9. Subqueries
10. IN subqueries
11. EXISTS
12. NOT EXISTS
13. CASE
14. COALESCE
15. NULL handling
16. Aggregate functions with joins
17. Correlated subqueries
18. Common table expressions
19. String manipulation
20. Date calculations

Include approximately 40-50 intermediate questions. Advanced Topics should include:

1. Window functions
2. ROW_NUMBER
3. RANK
4. DENSE_RANK
5. PARTITION BY
6. Running totals
7. Moving averages
8. LAG
9. LEAD
10. FIRST_VALUE
11. LAST_VALUE
12. CTEs
13. Recursive CTEs
14. Complex subqueries
15. Correlated subqueries
16. Multiple-level aggregations
17. Top-N-per-group problems
18. Second/third/Nth highest salary problems
19. Duplicate detection
20. Gaps and islands
21. Advanced date problems
22. Conditional aggregation
23. Complex joins
24. Query optimization concepts

Include approximately 40-50 advanced questions. 4. Practice Database Create realistic databases rather than toy tables. Use multiple related datasets. Dataset 1: Healthcare Tables: patients

- patient_id
- first_name
- last_name

- birth_date
- gender
- city
- province_id
- allergies

doctors

- doctor_id
- first_name
- last_name
- specialty
- hospital_id

admissions

- admission_id
- patient_id
- doctor_id
- admission_date
- discharge_date
- diagnosis

province_names

- province_id
- province_name

hospitals

- hospital_id
- hospital_name
- city

Dataset 2: E-commerce Tables: customers

- id
- name
- email
- city
- country
- joined_date

orders

- id
- customer_id
- total
- status
- order_date

products

- id
- name
- category_id

- price

categories

- id
- name

order_items

- id
- order_id
- product_id
- quantity
- price

Dataset 3: Company / Employees Tables: employees

- employee_id
- first_name
- last_name
- email
- dept_id
- manager_id
- salary
- hire_date

departments

- dept_id
- dept_name

This dataset must contain:

- Employees with managers
- Employees without managers
- Multiple employees in the same department
- Duplicate salaries
- Unique salaries
- Multiple employees with the same email for duplicate-data exercises
- Departments with different employee counts
- At least one department with no employees

This dataset should specifically support questions involving:

- Self joins
- Highest salary
- Second highest salary
- Nth highest salary
- Top 3 salaries per department
- Employees earning above department average
- Ranking
- Dense ranking
- Manager/employee relationships

5. SQL Engine The website must have a real SQL execution engine. Prefer: DuckDB-WASM running entirely in the browser. The user should be able to execute SQL without requiring a backend database server. If DuckDB-WASM is unsuitable for a specific feature, use another browser-compatible SQL engine, but prioritize:

- Real SQL execution
- Fast execution
- No server dependency
- Safe sandboxing
- Persistent practice data
- Good support for joins, subqueries, CTEs and window functions

The SQL editor should support syntax highlighting and preferably autocomplete. Use Monaco Editor if practical. 6. SQL Dialect Clearly display the SQL dialect being used. Prefer: SQL / DuckDB because the browser-based engine must behave consistently. Do not pretend the platform supports MySQL or Oracle-specific syntax unless it actually does. If a feature differs across SQL dialects, show a small note explaining the difference. 7. Question Interface Each question should have this layout: Left side Question panel containing:

- Level
- Topic
- Difficulty
- Question
- Requirements
- Relevant table schema
- Sample table preview
- Helpful hints

Example: Find the second highest salary in each department. Show:

employees

---

employee_id

first_name

last_name

dept_id

salary

manager_id

hire_date

Allow users to expand/collapse the schema. Right side SQL editor:

-- Write your SQL query here

SELECT ...

Buttons:

- Run Query
- Reset
- Clear
- Show Hint
- Show Solution

Do NOT show the solution by default. 8. Result Area After clicking Run Query, show: Query result A clean table displaying the returned rows. Also show:

- Number of rows returned
- Execution time
- Whether the query passed validation

Example:

✓ Correct

12 rows returned

Execution time: 14ms

If incorrect:

✗ Not quite

Your query executed successfully, but the result doesn't match

the expected result.

Try checking your filtering or grouping logic.

Do not simply say "Wrong." Give useful but not overly revealing feedback. 9. Query Validation The platform should validate the user's query based primarily on the result set, not exact SQL text. For example, these should both be accepted if they produce the correct result:

SELECT ...

and an alternative logically equivalent query. Validation should account for:

- Correct columns

- Correct values
- Correct row count
- Correct ordering when ordering is explicitly required

Avoid requiring one exact SQL implementation unless the exercise specifically teaches a particular technique. For example, if the question says: "Use an IN subquery" then validate that the user actually uses the intended concept if technically feasible. 10. Hints System Each question should have progressive hints. For example: Hint 1 "Think about what needs to be grouped." Hint 2 "The phrase 'each department' is a clue." Hint 3 "Consider PARTITION BY dept_id." Hints should become progressively more specific. Do not immediately reveal the complete query. 11. Solution System The user can click: Show Solution Then display:

1. Correct SQL query
2. Explanation
3. Step-by-step thought process
4. Alternative solution if applicable
5. Key concept learned

Example:

Concept:

DENSE_RANK + PARTITION BY

Thought process:

"Each department" → PARTITION BY dept_id

"Highest" → ORDER BY salary DESC

"Second" → rank = 2

This is important. The platform should teach users how to think about SQL questions, not just provide answers. 12. Schema Explorer Create a persistent database/schema explorer. Example:

DATABASE

│

├── patients

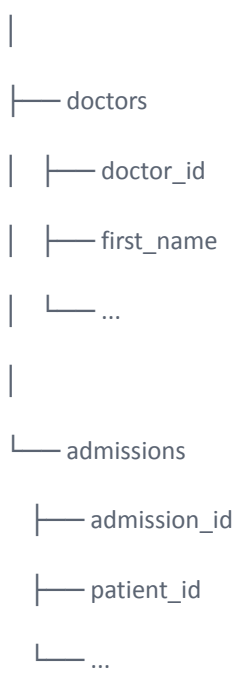
│ ├── patient_id

│ ├── first_name

│ ├── last_name

│ ├── birth_date

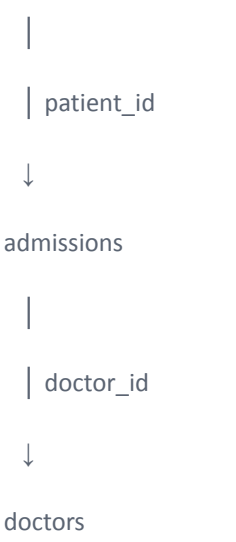
│ └── ...



Clicking a table should show:

- Columns
- Data types
- Sample rows
- Primary key
- Foreign keys
- Relationships

13. Database Relationships Visually show relationships. Example:



This should help beginners understand JOINs. 14. Practice Modes Create multiple practice modes. Guided Practice Shows:

- Question
- Schema
- Hints
- Concept explanation

Interview Mode Shows:

- Question only
- Timer
- No hints by default
- Score at the end

Random Practice Randomly selects questions based on:

- Level
- Topic
- Difficulty

Topic Practice Allow users to choose:

JOINs

GROUP BY

Subqueries

Window Functions

CTEs

Aggregations

15. Difficulty System Each question should have:

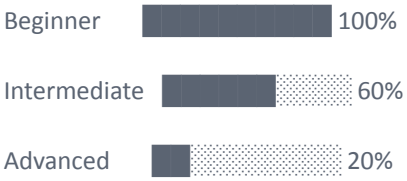
- Beginner
- Easy
- Medium
- Hard
- Expert

Difficulty should be based on reasoning complexity, not just number of SQL keywords. 16. Progress Tracking Store progress locally using browser storage. Track:

- Questions attempted
- Questions solved
- Questions failed
- Success rate
- Current level
- Topic strengths
- Topic weaknesses
- Streak
- Average solving time

Example dashboard:

SQL Progress



Questions solved: 84

Success rate: 76%

Current streak: 7 days

No login should be required for the first version. 17. Learning Path Create a recommended progression:

SQL BEGINNER

↓

SELECT

↓

WHERE

↓

ORDER BY

↓

GROUP BY

↓

HAVING

↓

JOINS

↓

SUBQUERIES

↓

CTEs

↓

WINDOW FUNCTIONS

↓

ADVANCED SQL

↓

INTERVIEW MODE

Lock advanced sections initially, but allow users to unlock them after completing prerequisite topics. Also allow users to manually unlock everything. 18. SQL Playground Create a separate page: SQL Playground Here users can freely experiment with the built-in databases. Features:

- Database selector
- Schema explorer
- SQL editor
- Run button
- Results
- Query history
- Reset database
- Sample queries

Users should be able to run arbitrary queries against the practice database. 19. Query History Store recent queries locally. Show:

Recent Queries

SELECT * FROM employees;

SELECT dept_id, AVG(salary) ...

SELECT ...

Allow:

- Re-run
- Copy
- Delete

20. Error Handling SQL errors should be displayed clearly. Instead of showing a raw technical error only:

Binder Error...

show:

SQL Error

Column "dept" does not exist.

Available columns:

dept_id

first_name

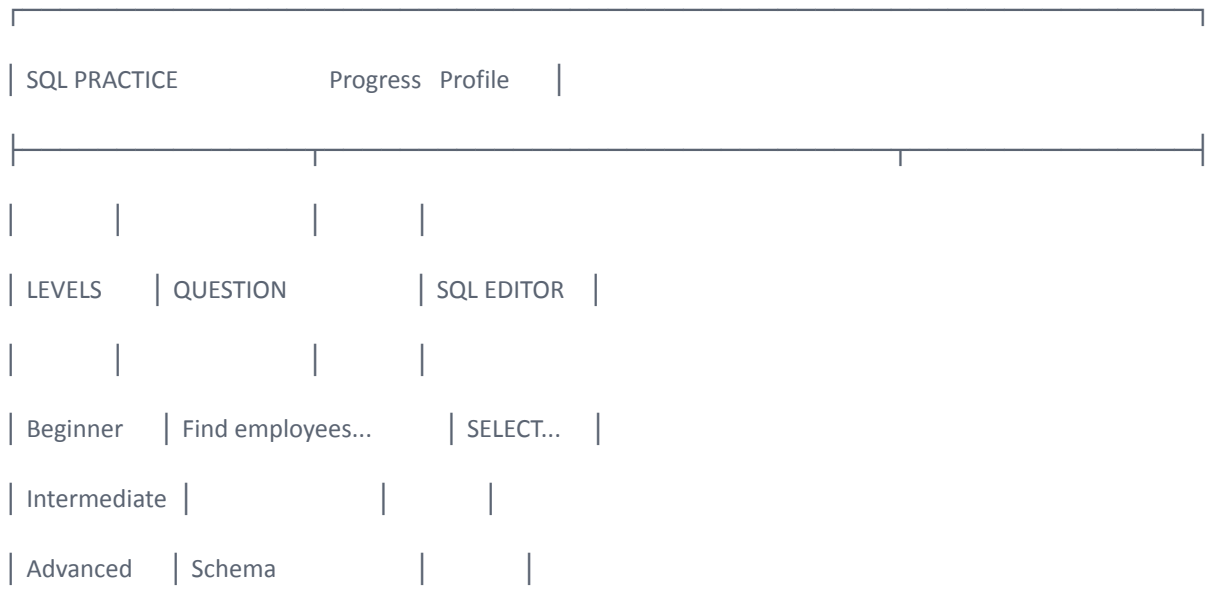
last_name

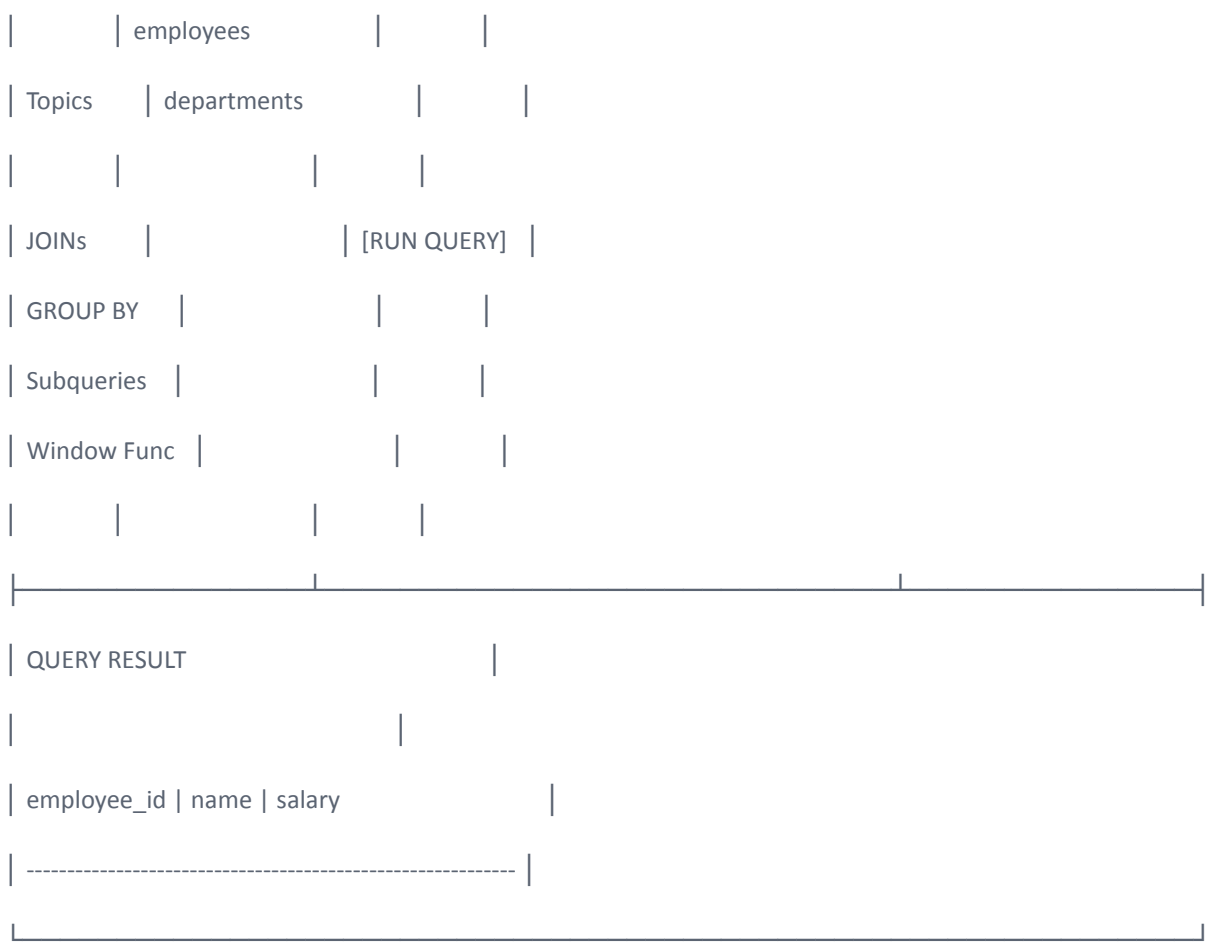
salary

But also allow the user to expand Technical Error for the raw database error. 21. UI / UX The design should feel like a modern developer learning platform. Use:

- Dark mode by default
- Clean typography
- Excellent code editor
- Minimal distractions
- Responsive layout
- Desktop-first design
- Mobile-friendly question browsing

Suggested layout:





22. Landing Page Create a polished landing page. Headline: Master SQL by Practicing. Subheadline: Go from your first SELECT query to advanced SQL interview problems using real datasets and an in-browser SQL engine. Primary CTA: Start Practicing Secondary CTA: Explore SQL Topics Show three cards:

BEGINNER

Build your SQL fundamentals

INTERMEDIATE

Master JOINs, subqueries and aggregation

ADVANCED

Solve window functions and interview-level problems

23. Question Content Do not create generic questions only. Questions should resemble actual SQL interview and practical problems. Examples: Beginner Show all patients born in 2010. Find cities that have more than 5 patients. Show unique first names that occur only once. Intermediate Find customers who have never placed an order. Find customers who have placed at least one order using an IN subquery. Find patients admitted multiple times for the same diagnosis. Advanced Find the second-highest salary in each department. Find the top 3 salaries in each department. Find employees earning more than their department average. Find the highest-paid employee in each department. Find employees who earn more than their manager. Find departments whose average salary is greater than the company average. Create enough data so these questions require actual SQL reasoning.
24. Dataset Quality Make the datasets realistic. Include:

- NULL values
- Duplicate values
- Duplicate names
- Duplicate salaries
- Missing relationships
- Customers with zero orders
- Customers with multiple orders
- Employees without managers
- Employees with the same salary
- Departments with many employees
- Departments with no employees
- Different dates
- Different categories

Do not make the dataset so small that every query becomes obvious. 25. Technology Use a modern frontend stack. Preferred:

- React
- TypeScript
- Vite or Next.js
- Tailwind CSS
- Monaco Editor
- DuckDB-WASM

Use clean component architecture. Separate:

components/

pages/

data/

questions/

sql-engine/

validation/

utils/

types/

Question data should be stored separately from UI code. 26. Question Data Structure Create a reusable structure similar to:

{

id: "advanced-023",

level: "advanced",

topic: "window-functions",

difficulty: "hard",

title: "Second highest salary in each department",

description: "...",

tables: ["employees"],

hints: [

"...",

"...",

"..."

],

solution: "...",

explanation: "...",

expectedResult: [...],

tags: ["dense-rank", "partition-by"]

}

The application should load questions dynamically from this structure. 27. Important Technical Requirement The SQL engine, schema creation and sample data should initialize automatically when the application loads. The user should NOT need to manually run:

CREATE TABLE ...

INSERT INTO ...

The platform handles database initialization. The user only writes queries. 28. Reset Functionality Provide: Reset Database This should restore the original dataset. Also provide: Reset Question This should clear the user's SQL editor for that question. 29. Accessibility Include:

- Keyboard navigation

- Good contrast
- Accessible buttons
- Proper labels
- Screen-reader-friendly structure
- Code editor keyboard shortcuts

30. Performance The application should feel instant. Since the SQL engine runs locally:

- Avoid unnecessary network requests
- Cache initialized databases
- Keep question data lightweight
- Prevent the UI from freezing during query execution
- Handle malformed or expensive queries safely

31. No Backend Authentication in V1 Do not add authentication, payments, subscriptions or user accounts initially. Everything should work immediately after opening the website. Use localStorage/IndexedDB for progress. The architecture should make it possible to add authentication later.

32. Final Deliverable Build the complete working website, not just a mockup. I should be able to:

33. Open the website

34. Select Beginner

35. Open a question

36. See the schema

37. Write SQL

38. Run SQL

39. See results

40. Get pass/fail feedback

41. Get hints

42. View the solution

43. Move to the next question

44. Track progress

45. Practice intermediate and advanced problems

46. Open the free SQL playground

47. Run arbitrary SQL against the built-in datasets

The application must be functional end-to-end. Do not leave placeholder buttons or fake SQL execution. Before finishing, test the complete user flow and make sure the included questions, schemas, expected results and SQL engine all work correctly.

2nd Update - the prompt used 👍

Add a PDF-Based SQL Practice Section to the Existing Website The SQL practice website is already built and working. I want to add a new tab/section called "PDF Practice" without breaking or changing the existing functionality. The purpose of this section is: I should be able to upload a PDF containing SQL database schemas, sample data, questions, and answers. The website should use the PDF as the source material and convert it into an interactive SQL practice experience.

1. New Navigation Tab Add a new main navigation item: PDF Practice Existing sections of the website must continue working exactly as they do. Do not redesign or remove existing functionality unless required for integration.
2. PDF Upload Page When the user opens PDF Practice, show: Upload area

Support:

- PDF upload
- Drag and drop
- Multiple PDFs
- Remove uploaded PDF
- Re-upload PDF
- Clear/reset PDF

After uploading, process the PDF automatically. 3. PDF Processing The application should extract the following information from the PDF: Database schema Identify:

- Table names
- Column names
- Data types where available
- Primary keys
- Foreign keys
- Relationships
- Sample data if provided

Example:

employees

employee_id
first_name
last_name
dept_id
salary

manager_id
hire_date

Questions Identify every SQL practice question from the PDF. Example: Find the second highest salary in each department. Answers Identify the corresponding answer/query if the PDF contains one. Example:

SELECT ...

The extracted questions and answers should maintain their original ordering. 4. Create a Practice Set After processing the PDF, show a summary:

PDF Processed Successfully

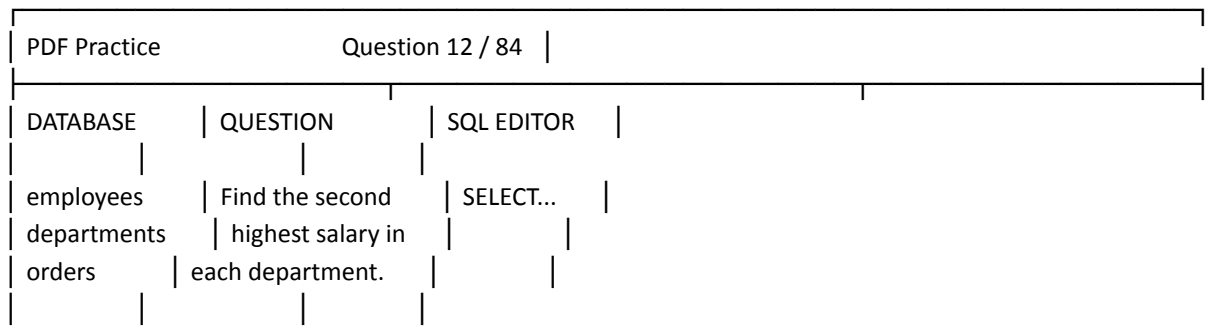
Tables: 6
Questions: 84
Answers found: 84

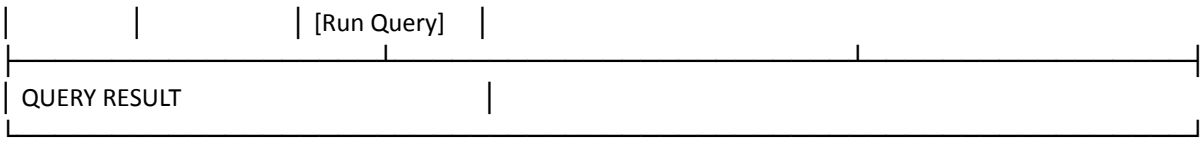
[Start Practice]

Also show:

Database
Questions
Progress

5. Automatically Create the SQL Database If the PDF contains SQL `CREATE TABLE` and `INSERT` statements, use them to initialize the SQL engine. If the PDF contains schema/data in a readable tabular format instead of SQL statements, convert the information into the internal database structure. Use the same SQL engine already used by the website. Do NOT create a separate SQL engine just for PDF Practice. The PDF Practice section should use the existing SQL execution engine.
6. Question Practice Interface After clicking Start Practice, show an interface similar to the existing practice page. Example:





Reuse existing UI components wherever possible. 7. Question Navigation Allow:

- Previous question
- Next question
- Question number navigation
- Jump to any question
- Mark question as completed
- Mark question for review

Example:

Questions

1 ✓
2 ✓
3 ✓
4 ○
5 🔖
6 ○
7 ○
...

Legend:

✓ = completed
○ = not attempted
🔖 = marked for review

8. Show the Original Question Display the question exactly as extracted from the PDF. Do not unnecessarily rewrite the question. If the PDF contains: Show the total amount of male patients and the total amount of female patients in the patients table. Display that question.
9. Schema Visibility For every question, allow the user to inspect the relevant database tables. Provide: Schema

patients
├── patient_id
├── first_name
├── last_name
├── birth_date
├── gender

```
├── city
└── allergies
```

Allow clicking a table to see sample rows. 10. SQL Editor Use the existing Monaco SQL editor. Provide:

```
-- Write your SQL query here
```

Buttons:

- Run Query
- Clear
- Reset
- Hint
- Show Answer

Do not show the answer automatically. 11. Query Validation This is extremely important. The PDF may contain the expected SQL answer, but do not simply compare the user's SQL text against the PDF answer. Instead, execute the user's query and compare the result against the expected result. For example, if the PDF answer is:

```
SELECT ...
```

execute that answer against the database to generate the expected result. Then execute the user's query. Compare:

- Columns
- Values
- Number of rows
- Ordering when ordering is part of the question

Accept logically equivalent SQL queries when they produce the correct result. 12. If the PDF Contains Answers but No Expected Results Automatically execute the extracted answer against the initialized database. Store the resulting dataset as the expected result for that question. Example internal structure:

```
{
  questionId: "pdf-12",
  question: "...",
  answerSql: "...",
  expectedResult: [...]
}
```

This allows validation without requiring manually entered expected results. 13. If the PDF Contains Questions but No Answers Still allow the user to practice the question. Show:

⚠ Answer not available in source PDF

But allow the user to execute SQL normally. Do not invent an answer. 14. Hints Generate hints based on the question and extracted answer. However, hints must not immediately reveal the complete answer. Example: Question: Find the second highest salary in each department. Hint 1: Think about how "each department" affects the calculation. Hint 2: Consider a window function. Hint 3: Look into PARTITION BY. Only show the complete SQL when the user clicks: Show Answer 15. Answer Display When the user clicks Show Answer, display: Correct SQL

SELECT ...

Explanation Explain:

- Why the query works
- Important SQL concepts
- How the tables are connected
- Why specific clauses/functions were used

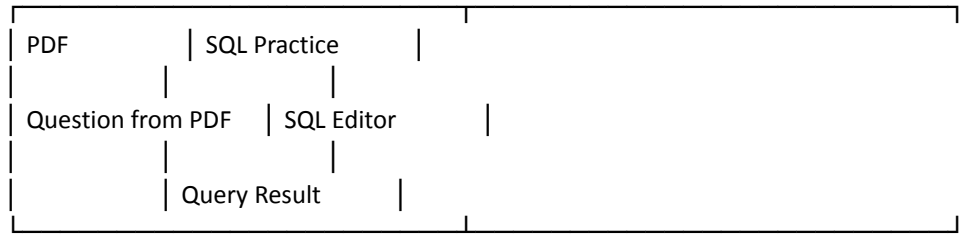
Also show: Key concepts

DENSE_RANK
PARTITION BY
ORDER BY
Subquery

16. PDF Reference Panel Add an option: View PDF The user should be able to open the original PDF while practicing. Prefer a split-screen or modal PDF viewer. Example:

[SQL Practice] [View PDF]

When the PDF viewer opens:



This allows users to refer back to the original material. 17. Search Questions Add search functionality. Example:

Search questions...

Allow searches such as:

JOIN
GROUP BY
salary
subquery
patients
window functions

Filter questions based on extracted text. 18. Filter Questions Allow filtering by:

- All
- Unattempted
- Completed
- Incorrect
- Marked for review

If the PDF contains identifiable SQL topics, also allow:

- SELECT
- WHERE
- GROUP BY
- HAVING
- JOIN
- Subquery
- CTE
- Window Functions
- etc.

If topics cannot reliably be extracted, don't invent them. 19. Progress Tracking Track progress separately for each uploaded PDF. Example:

SQL Interview Questions.pdf

Progress
60%

48 / 80 completed

Correct: 39
Incorrect: 9
Unattempted: 32

Store progress locally using IndexedDB/localStorage. If the user closes the browser and comes back, progress should remain. 20. Multiple PDFs Support multiple uploaded PDFs. Example:

My Practice PDFs

SQL Interview Questions
84 questions
72% completed

Healthcare SQL
42 questions
100% completed

Advanced SQL
60 questions
24% completed

Clicking a PDF opens its own practice set. 21. Persistent PDF Storage Prefer IndexedDB for storing uploaded PDFs and extracted practice data. Do not store large PDFs directly in localStorage. The user should be able to:

- Upload PDF
- Rename practice set
- Delete PDF
- Reopen PDF later
- Continue progress

22. Import Processing Status When uploading a large PDF, show progress. Example:

Processing PDF...

✓ Reading PDF
✓ Extracting database schema
✓ Extracting questions
✓ Extracting answers
⏳ Building SQL database
○ Validating questions

72%

Do not make the interface appear frozen. 23. Extraction Robustness PDFs may have different formats. The extraction system should handle: Format A Question followed by answer:

Question:
Find all employees...

Answer:
SELECT ...

Format B Questions numbered:

1. Find all employees...
2. Find the highest salary...
3. Find...

with answers later. Format C SQL schema:

CREATE TABLE...
INSERT INTO...

Format D Tables shown visually in the PDF. Try to detect the structure intelligently. If extraction confidence is low, show a review screen rather than silently creating incorrect questions. 24. Import Review Screen Before creating the practice set, show:

Review Imported Content

Tables detected: 5
Questions detected: 73
Answers detected: 69

⚠ 4 questions do not have answers.

[Review Questions]

[Create Practice Set]

Allow the user to inspect extracted questions before finalizing. 25. Data Safety The PDF should be processed locally in the browser whenever practical. Do not upload the user's PDF to an external service unless absolutely necessary. The SQL database should also remain local to the browser. 26. Reuse Existing Architecture Do not duplicate existing SQL engine functionality. The new PDF Practice section should reuse:

- Existing SQL engine
- Existing Monaco editor
- Existing query execution
- Existing result table
- Existing styling system
- Existing progress components
- Existing navigation components

Create reusable components only where necessary. 27. Important UX Principle The PDF is the source material. The website

should transform it into a structured practice experience. The user should NOT have to manually:

- Copy questions
- Copy database schema
- Create tables
- Insert data
- Copy answers
- Create expected results

The application should automate these steps as much as possible. 28. Final User Flow The complete flow should be:

Open Website
↓
PDF Practice
↓
Upload PDF
↓
Extract schema + data + questions + answers
↓
Review extracted content
↓
Create Practice Set
↓
Initialize SQL database
↓
Start Practice
↓
Read Question
↓
Write SQL
↓
Run Query
↓
Compare Result
↓
✓ Correct / ✗ Incorrect
↓
Hint if needed
↓
Show Answer if needed
↓
Next Question
↓
Track Progress

29. Build Requirements Implement this as a real working feature. Do NOT create a static mockup. The following must actually work:

- PDF upload

- PDF parsing
- Question extraction
- Answer extraction
- Schema/data extraction
- Database initialization
- SQL execution
- Query validation
- Hints
- Answer display
- PDF viewer
- Question navigation
- Search
- Filtering
- Progress tracking
- Multiple PDFs
- Persistent storage

Before finishing, test the feature using at least one real SQL practice PDF containing:

- Multiple tables
- SQL schema
- Sample data
- At least 10 questions
- Answers

Fix extraction and validation issues discovered during testing. Do not break any existing functionality of the website.